

Лістинг коду програми

Вихідні файли також доступні за посиланням <https://github.com/Teollan/2026-comparch-lab3>.

apps/app.cpp

```
1  #include <Args.hpp>
2  #include <Decompiler.hpp>
3  #include <Hex.hpp>
4  #include <Instruction.hpp>
5  #include <cstdint>
6  #include <fstream>
7  #include <iostream>
8  #include <iterator>
9  #include <sstream>
10 #include <vector>
11
12 int main(int argc, char *argv[]) {
13     Args args(argc, argv);
14     args.expectString("input", "i", true)
15         .expectFlag("hex", "x")
16         .expectString("output", "o");
17
18     try {
19         args.parse();
20     } catch (const ArgsException &e) {
21         std::cerr << e.what() << "\n";
22         args.printUsage();
23         return 1;
24     }
25
26     std::ifstream input(args.getString("input"), std::ios::binary);
27     if (!input) {
28         std::cerr << "Failed to open input: " << args.getString("input")
29             << "\n";
30         return 1;
31     }
32
33     std::vector<std::uint8_t> bytes;
34     if (args.getFlag("hex")) {
35         std::stringstream buffer;
36
37         buffer << input.rdbuf();
38
39         bytes = hexToBytes(buffer.str());
40     } else {
41         bytes.assign(
42             std::istreambuf_iterator<char>(input),
43             std::istreambuf_iterator<char>()
44         );
45     }
46
47     auto instructions = decompile(bytes);
48
49     std::ofstream outputFile;
50     std::ostream *out = &std::cout;
51     if (!args.getString("output").empty()) {
```

```
52     outputFile.open(args.getString("output"));
53
54     if (!outputFile) {
55         std::cerr << "Failed to open output: " << args.getString("output")
56             << "\n";
57         return 1;
58     }
59
60     out = &outputFile;
61 }
62
63 for (const auto &instruction : instructions) {
64     *out << bytesToHex(instruction.bytes) << "\n"
65         << instruction.toAsm() << "\n\n";
66 }
67
68 return 0;
69 }
```

```
1 #pragma once
2
3 #include <stdexcept>
4 #include <string>
5 #include <unordered_map>
6 #include <vector>
7
8 class ArgsException : public std::runtime_error {
9 public:
10     using std::runtime_error::runtime_error;
11 };
12
13 class Args {
14 private:
15     struct StringArg {
16         std::string shortName;
17         bool required;
18         std::string value;
19         bool provided = false;
20     };
21
22     struct FlagArg {
23         std::string shortName;
24         bool value = false;
25     };
26
27     std::string _program;
28     std::vector<std::string> _argv;
29
30     std::unordered_map<std::string, StringArg> _strings;
31     std::unordered_map<std::string, FlagArg> _flags;
32     std::unordered_map<std::string, std::string> _aliases;
33
34 public:
35     Args(int argc, char *argv[]);
36
37     Args &expectString(
38         const std::string &name,
39         const std::string &shortName,
40         bool required = false
41     );
42     Args &expectFlag(const std::string &name, const std::string &shortName);
43
44     Args &parse();
45
46     void printUsage() const;
47
48     [[nodiscard]] const std::string &getString(const std::string &name) const;
49     [[nodiscard]] bool getFlag(const std::string &name) const;
50 };
```

```
1 #include "Args.hpp"
2
3 #include <format>
4 #include <iostream>
5 #include <stdexcept>
6
7 Args::Args(int argc, char *argv[]) : _program(argv[0]) {
8     for (int i = 1; i < argc; i++) {
9         _argv.emplace_back(argv[i]);
10    }
11 }
12
13 Args &Args::expectString(
14     const std::string &name, const std::string &shortName, bool required
15 ) {
16     _strings[name] = {shortName, required, "", false};
17     _aliases["--" + name] = name;
18     _aliases["-" + shortName] = name;
19
20     return *this;
21 }
22
23 Args &Args::expectFlag(const std::string &name, const std::string &shortName) {
24     _flags[name] = {shortName, false};
25     _aliases["--" + name] = name;
26     _aliases["-" + shortName] = name;
27
28     return *this;
29 }
30
31 Args &Args::parse() {
32     for (size_t i = 0; i < _argv.size(); i++) {
33         const auto &arg = _argv[i];
34
35         auto alias = _aliases.find(arg);
36
37         if (alias == _aliases.end()) {
38             throw ArgsException("Unknown argument: " + arg);
39         }
40
41         const auto &name = alias->second;
42
43         auto stringArg = _strings.find(name);
44         if (stringArg != _strings.end()) {
45             if (i + 1 >= _argv.size()) {
46                 throw ArgsException("Missing value for: " + arg);
47             }
48
49             stringArg->second.value = _argv[++i];
50             stringArg->second.provided = true;
51
52             continue;
53         }
54
55         auto flagArg = _flags.find(name);
56         if (flagArg != _flags.end()) {
```

```

57         flagArg->second.value = true;
58     }
59 }
60
61 for (const auto &[name, arg] : _strings) {
62     if (arg.required && !arg.provided) {
63         throw ArgsException("Missing required argument: --" + name);
64     }
65 }
66
67 return *this;
68 }
69
70 void Args::printUsage() const {
71     std::cerr << "\nUsage: " << _program;
72
73     for (const auto &[name, arg] : _strings) {
74         if (arg.required) {
75             std::cerr << " --" << name << " <value>";
76         }
77     }
78
79     std::cerr << " [options]\n\nOptions:\n";
80
81     for (const auto &[name, arg] : _strings) {
82         std::cerr << std::format(
83             "  --{}, -{} <value>{}\n",
84             name,
85             arg.shortName,
86             arg.required ? " (required)" : ""
87         );
88     }
89
90     for (const auto &[name, arg] : _flags) {
91         std::cerr << std::format("  --{}, -{}\n", name, arg.shortName);
92     }
93 }
94
95 const std::string &Args::getString(const std::string &name) const {
96     auto it = _strings.find(name);
97
98     if (it == _strings.end()) {
99         throw std::runtime_error("Unknown string argument: " + name);
100     }
101
102     return it->second.value;
103 }
104
105 bool Args::getFlag(const std::string &name) const {
106     auto it = _flags.find(name);
107
108     if (it == _flags.end()) {
109         throw std::runtime_error("Unknown flag argument: " + name);
110     }
111
112     return it->second.value;
113 }

```

```
1 #include "Args.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <string>
6 #include <vector>
7
8 namespace {
9
10 class MockArgv {
11 public:
12     explicit MockArgv(std::initializer_list<std::string> tokens) :
13         _tokens(tokens) {
14         for (auto &s : _tokens) {
15             _ptrs.push_back(s.data());
16         }
17     }
18
19     int argc() const {
20         return static_cast<int>(_ptrs.size());
21     }
22
23     char **argv() {
24         return _ptrs.data();
25     }
26
27 private:
28     std::vector<std::string> _tokens;
29     std::vector<char *> _ptrs;
30 };
31
32 } // namespace
33
34 TEST(Args, ParsesRequiredStringArg) {
35     MockArgv input{"/app", "--input", "foo.bin"};
36     Args args(input.argc(), input.argv());
37
38     args.expectString("input", "i", true).parse();
39
40     EXPECT_EQ(args.getString("input"), "foo.bin");
41 }
42
43 TEST(Args, ParsesShortFormStringArg) {
44     MockArgv input{"/app", "-i", "foo.bin"};
45     Args args(input.argc(), input.argv());
46
47     args.expectString("input", "i", true).parse();
48
49     EXPECT_EQ(args.getString("input"), "foo.bin");
50 }
51
52 TEST(Args, ParsesFlagLongForm) {
53     MockArgv input{"/app", "--hex"};
54     Args args(input.argc(), input.argv());
55
56     args.expectFlag("hex", "x").parse();
```

```

57
58     EXPECT_TRUE(args.getFlag("hex"));
59 }
60
61 TEST(Args, ParsesFlagShortForm) {
62     MockArgv input{"/app", "-x"};
63     Args args(input.argc(), input.argv());
64
65     args.expectFlag("hex", "x").parse();
66
67     EXPECT_TRUE(args.getFlag("hex"));
68 }
69
70 TEST(Args, UnsetFlagDefaultsToFalse) {
71     MockArgv input{"/app"};
72     Args args(input.argc(), input.argv());
73
74     args.expectFlag("hex", "x").parse();
75
76     EXPECT_FALSE(args.getFlag("hex"));
77 }
78
79 TEST(Args, OptionalStringDefaultsToEmpty) {
80     MockArgv input{"/app"};
81     Args args(input.argc(), input.argv());
82
83     args.expectString("output", "o").parse();
84
85     EXPECT_EQ(args.getString("output"), "");
86 }
87
88 TEST(Args, MixesStringsAndFlags) {
89     MockArgv input{"/app", "-i", "in.bin", "-x", "-o", "out.txt"};
90     Args args(input.argc(), input.argv());
91
92     args.expectString("input", "i", true)
93         .expectFlag("hex", "x")
94         .expectString("output", "o")
95         .parse();
96
97     EXPECT_EQ(args.getString("input"), "in.bin");
98     EXPECT_TRUE(args.getFlag("hex"));
99     EXPECT_EQ(args.getString("output"), "out.txt");
100 }
101
102 TEST(Args, OrderOfArgsDoesNotMatter) {
103     MockArgv input{"/app", "-o", "out.txt", "--hex", "--input", "in.bin"};
104     Args args(input.argc(), input.argv());
105
106     args.expectString("input", "i", true)
107         .expectFlag("hex", "x")
108         .expectString("output", "o")
109         .parse();
110
111     EXPECT_EQ(args.getString("input"), "in.bin");
112     EXPECT_TRUE(args.getFlag("hex"));
113     EXPECT_EQ(args.getString("output"), "out.txt");
114 }
115
116 TEST(Args, ThrowsOnMissingRequired) {

```

```
117     MockArgv input{"/app"};
118     Args args(input.argc(), input.argv());
119
120     args.expectString("input", "i", true);
121
122     EXPECT_THROW(args.parse(), ArgsException);
123 }
124
125 TEST(Args, ThrowsOnUnknownArg) {
126     MockArgv input{"/app", "--bogus"};
127     Args args(input.argc(), input.argv());
128
129     EXPECT_THROW(args.parse(), ArgsException);
130 }
131
132 TEST(Args, ThrowsOnMissingValueForStringArg) {
133     MockArgv input{"/app", "--input"};
134     Args args(input.argc(), input.argv());
135
136     args.expectString("input", "i", true);
137
138     EXPECT_THROW(args.parse(), ArgsException);
139 }
140
141 TEST(Args, GetStringThrowsOnUnknownName) {
142     MockArgv input{"/app"};
143     Args args(input.argc(), input.argv());
144
145     args.parse();
146
147     EXPECT_THROW(args.getString("never-registered"), std::runtime_error);
148 }
149
150 TEST(Args, GetFlagThrowsOnUnknownName) {
151     MockArgv input{"/app"};
152     Args args(input.argc(), input.argv());
153
154     args.parse();
155
156     EXPECT_THROW(args.getFlag("never-registered"), std::runtime_error);
157 }
158
159 TEST(Args, LaterValueOverridesEarlier) {
160     MockArgv input{"/app", "-i", "first.bin", "-i", "second.bin"};
161     Args args(input.argc(), input.argv());
162
163     args.expectString("input", "i", true).parse();
164
165     EXPECT_EQ(args.getString("input"), "second.bin");
166 }
```

```
1 #pragma once
2
3 #include <Instruction.hpp>
4 #include <cstdint>
5 #include <vector>
6
7 std::vector<Instruction> decompile(std::vector<std::uint8_t> &bytes);
8
9 class DecompilerUnknownInstructionException : public std::exception {
10 private:
11     std::size_t offset;
12
13 public:
14     DecompilerUnknownInstructionException(std::size_t offset);
15 };
```

```
1  #include "Decompiler.hpp"
2
3  std::vector<Instruction> decompile(std::vector<std::uint8_t> &bytes) {
4      std::vector<Instruction> instructions;
5
6      auto iterator = bytes.begin();
7      while (iterator != bytes.end()) {
8          std::optional<Instruction> instruction = std::nullopt;
9
10         for (const auto &prototype : INSTRUCTION_PROTOTYPES) {
11             auto instructionBytes = std::vector<std::uint8_t>(
12                 iterator, iterator + prototype.size()
13             );
14
15             if (prototype.doesMatch(instructionBytes)) {
16                 instruction.emplace(prototype, instructionBytes);
17
18                 break;
19             }
20         }
21
22         if (!instruction) {
23             throw DecompilerUnknownInstructionException(
24                 iterator - bytes.begin()
25             );
26         }
27
28         instructions.push_back(instruction.value());
29         iterator += instruction->bytes.size();
30     };
31
32     return instructions;
33 }
34
35 DecompilerUnknownInstructionException::DecompilerUnknownInstructionException(
36     std::size_t offset
37 ) :
38     offset(offset) {}
```

```
1 #include "Decompiler.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <vector>
6
7 using Bytes = std::vector<std::uint8_t>;
8
9 TEST(decompile, EmptyInputReturnsEmptyResult) {
10     Bytes bytes;
11     auto result = decompile(bytes);
12
13     EXPECT_TRUE(result.empty());
14 }
15
16 TEST(decompile, SingleTwoByteInstruction) {
17     Bytes bytes{0x0A, 0x12};
18     auto result = decompile(bytes);
19
20     ASSERT_EQ(result.size(), 1);
21     EXPECT_EQ(result[0].bytes, bytes);
22     EXPECT_EQ(result[0].prototype.opcode, 0x0A);
23     EXPECT_EQ(result[0].toAsm(), "SUB R1, R2");
24 }
25
26 TEST(decompile, SingleThreeByteInstruction) {
27     Bytes bytes{0x03, 0x01, 0x23};
28     auto result = decompile(bytes);
29
30     ASSERT_EQ(result.size(), 1);
31     EXPECT_EQ(result[0].prototype.opcode, 0x030);
32     EXPECT_EQ(result[0].toAsm(), "ADD R1, R2, R3");
33 }
34
35 TEST(decompile, SingleSixByteInstruction) {
36     Bytes bytes{0x1B, 0x01, 0x00, 0x00, 0x00, 0x42};
37     auto result = decompile(bytes);
38
39     ASSERT_EQ(result.size(), 1);
40     EXPECT_EQ(result[0].prototype.opcode, 0x1B0);
41     EXPECT_EQ(result[0].toAsm(), "MOV R1, [0x00000042]");
42 }
43
44 TEST(decompile, MultipleInstructionsSameSize) {
45     Bytes bytes{
46         0x0A,
47         0x12,
48         0x80,
49         0xAF,
50     };
51     auto result = decompile(bytes);
52
53     ASSERT_EQ(result.size(), 2);
54     EXPECT_EQ(result[0].toAsm(), "SUB R1, R2");
55     EXPECT_EQ(result[1].toAsm(), "CMP R10, R15");
56 }
```

```

57
58 TEST(decompile, MultipleInstructionsMixedSizes) {
59     Bytes bytes{
60         0x0A,
61         0x12,    // 2 bytes: SUB R1, R2
62         0x03,
63         0x01,
64         0x23,    // 3 bytes: ADD R1, R2, R3
65         0x1B,
66         0x01,
67         0x00,
68         0x00,
69         0x00,
70         0x42,    // 6 bytes: MOV R1, [...]
71         0x80,
72         0xAF,    // 2 bytes: CMP R10, R15
73     };
74     auto result = decompile(bytes);
75
76     ASSERT_EQ(result.size(), 4);
77     EXPECT_EQ(result[0].toAsm(), "SUB R1, R2");
78     EXPECT_EQ(result[1].toAsm(), "ADD R1, R2, R3");
79     EXPECT_EQ(result[2].toAsm(), "MOV R1, [0x00000042]");
80     EXPECT_EQ(result[3].toAsm(), "CMP R10, R15");
81 }
82
83 TEST(decompile, DistinguishesNibbleVariantOpcodes) {
84     // 1B 0 ... -> MOV reg, addr
85     // 1B 1 ... -> MOV addr, reg (operand order swapped)
86     Bytes bytes{
87         0x1B,
88         0x01,
89         0x00,
90         0x00,
91         0x00,
92         0x42,
93         0x1B,
94         0x12,
95         0x00,
96         0x00,
97         0x00,
98         0x42,
99     };
100     auto result = decompile(bytes);
101
102     ASSERT_EQ(result.size(), 2);
103     EXPECT_EQ(result[0].prototype.opcode, 0x1B0);
104     EXPECT_EQ(result[0].toAsm(), "MOV R1, [0x00000042]");
105     EXPECT_EQ(result[1].prototype.opcode, 0x1B1);
106     EXPECT_EQ(result[1].toAsm(), "MOV [0x00000042], R2");
107 }
108
109 TEST(decompile, DistinguishesLit16AndLit32) {
110     // 1C 1 ... -> 4-byte form (lit16)
111     // 1C 2 ... -> 6-byte form (lit32)
112     Bytes bytes{
113         0x1C,
114         0x11,
115         0x33,
116         0x44,

```

```
117         0x1C,  
118         0x21,  
119         0x33,  
120         0x44,  
121         0x55,  
122         0x66,  
123     };  
124     auto result = decompile(bytes);  
125  
126     ASSERT_EQ(result.size(), 2);  
127     EXPECT_EQ(result[0].prototype.opcode, 0x1C1);  
128     EXPECT_EQ(result[0].bytes.size(), 4);  
129     EXPECT_EQ(result[1].prototype.opcode, 0x1C2);  
130     EXPECT_EQ(result[1].bytes.size(), 6);  
131 }  
132  
133 TEST(decompile, ThrowsOnUnknownOpcode) {  
134     // 0xFF is not a recognized opcode prefix  
135     Bytes bytes{0xFF, 0x00};  
136  
137     EXPECT_THROW(decompile(bytes), DecompilerUnknownInstructionException);  
138 }  
139  
140 TEST(decompile, ThrowsAfterDecodingValidPrefix) {  
141     // Decode the SUB first, then choke on unknown 0xFF  
142     Bytes bytes{0x0A, 0x12, 0xFF, 0x00};  
143  
144     EXPECT_THROW(decompile(bytes), DecompilerUnknownInstructionException);  
145 }  
146  
147 TEST(decompile, BytesFieldOnInstructionMatchesInputSlice) {  
148     Bytes bytes{0x1B, 0x01, 0x00, 0x00, 0x00, 0x42, 0x0A, 0x12};  
149     auto result = decompile(bytes);  
150  
151     ASSERT_EQ(result.size(), 2);  
152     EXPECT_EQ(result[0].bytes, (Bytes{0x1B, 0x01, 0x00, 0x00, 0x00, 0x42}));  
153     EXPECT_EQ(result[1].bytes, (Bytes{0x0A, 0x12}));  
154 }
```

src/Hex/Hex.hpp

```
1 #pragma once
2
3 #include <cstdint>
4 #include <string>
5 #include <string_view>
6 #include <vector>
7
8 std::vector<std::uint8_t> hexToBytes(std::string_view hex);
9
10 std::string bytesToHex(const std::vector<std::uint8_t> &bytes);
```

```
1 #include "Hex.hpp"
2
3 #include <format>
4 #include <stdexcept>
5
6 namespace {
7
8 int hexDigitValue(char nibble) {
9     if (nibble >= '0' && nibble <= '9') {
10         return nibble - '0';
11     }
12
13     if (nibble >= 'A' && nibble <= 'F') {
14         return nibble - 'A' + 10;
15     }
16
17     if (nibble >= 'a' && nibble <= 'f') {
18         return nibble - 'a' + 10;
19     }
20
21     return -1;
22 }
23
24 } // namespace
25
26 std::vector<std::uint8_t> hexToBytes(std::string_view hex) {
27     std::vector<std::uint8_t> result;
28     int high = -1;
29
30     for (std::size_t i = 0; i < hex.size(); i++) {
31         if (hex[i] == '/' && i + 1 < hex.size() && hex[i + 1] == '/') {
32             while (i < hex.size() && hex[i] != '\n') {
33                 i++;
34             }
35
36             continue;
37         }
38
39         int value = hexDigitValue(hex[i]);
40
41         if (value < 0) {
42             continue;
43         }
44
45         if (high < 0) {
46             high = value;
47
48             continue;
49         }
50
51         result.push_back(static_cast<std::uint8_t>((high << 4) | value));
52         high = -1;
53     }
54
55     if (high >= 0) {
56         throw std::invalid_argument("hexToBytes: odd number of hex digits");
```

```
57     }
58
59     return result;
60 }
61
62 std::string bytesToHex(const std::vector<std::uint8_t> &bytes) {
63     std::string result;
64
65     for (std::size_t i = 0; i < bytes.size(); i++) {
66         result += std::format("{:02X} ", bytes[i]);
67     }
68
69     return result;
70 }
```

```
1 #include "Hex.hpp"
2
3 #include <gtest/gtest.h>
4
5 #include <stdexcept>
6
7 using Bytes = std::vector<std::uint8_t>;
8
9 TEST(hexToBytes, Empty) {
10     EXPECT_EQ(hexToBytes(""), Bytes{});
11 }
12
13 TEST(hexToBytes, WhitespaceOnly) {
14     EXPECT_EQ(hexToBytes("  \t\n  "), Bytes{});
15 }
16
17 TEST(hexToBytes, SingleByte) {
18     EXPECT_EQ(hexToBytes("0A"), Bytes{0x0A});
19     EXPECT_EQ(hexToBytes("FF"), Bytes{0xFF});
20 }
21
22 TEST(hexToBytes, CaseInsensitive) {
23     EXPECT_EQ(hexToBytes("ab"), Bytes{0xAB});
24     EXPECT_EQ(hexToBytes("Ab"), Bytes{0xAB});
25     EXPECT_EQ(hexToBytes("AB"), Bytes{0xAB});
26 }
27
28 TEST(hexToBytes, SpaceSeparated) {
29     EXPECT_EQ(hexToBytes("0A 12 80 AF"), (Bytes{0x0A, 0x12, 0x80, 0xAF}));
30 }
31
32 TEST(hexToBytes, NoSeparators) {
33     EXPECT_EQ(hexToBytes("0A1280AF"), (Bytes{0x0A, 0x12, 0x80, 0xAF}));
34 }
35
36 TEST(hexToBytes, MixedWhitespace) {
37     EXPECT_EQ(hexToBytes("0A\t12\n80\r\nAF"), (Bytes{0x0A, 0x12, 0x80, 0xAF}));
38 }
39
40 TEST(hexToBytes, OddDigitCountThrows) {
41     EXPECT_THROW(hexToBytes("A"), std::invalid_argument);
42     EXPECT_THROW(hexToBytes("0A 12 8"), std::invalid_argument);
43 }
44
45 TEST(hexToBytes, NonHexCharsAreSkipped) {
46     EXPECT_EQ(hexToBytes("0A, 12; 80 | AF"), (Bytes{0x0A, 0x12, 0x80, 0xAF}));
47 }
48
49 TEST(hexToBytes, LineCommentIsSkipped) {
50     EXPECT_EQ(
51         hexToBytes("0A 12 // SUB R1, R2\n80 AF // CMP R10, R15\n"),
52         (Bytes{0x0A, 0x12, 0x80, 0xAF})
53     );
54 }
55
56 TEST(hexToBytes, FullLineComment) {
```

```

57     EXPECT_EQ(
58         hexToBytes("// this is a header comment\n0A 12\n"),
59         (Bytes{0x0A, 0x12})
60     );
61 }
62
63 TEST(hexToBytes, CommentSkipsHexLikeChars) {
64     EXPECT_EQ(
65         hexToBytes("0A 12 // AF BE EF\n80 AF"),
66         (Bytes{0x0A, 0x12, 0x80, 0xAF})
67     );
68 }
69
70 TEST(hexToBytes, CommentAtEndOfInputWithoutNewline) {
71     EXPECT_EQ(hexToBytes("0A 12 // no trailing newline"), (Bytes{0x0A, 0x12}));
72 }
73
74 TEST(hexToBytes, SingleSlashIsNotAComment) {
75     EXPECT_EQ(hexToBytes("0A / 12"), (Bytes{0x0A, 0x12}));
76 }
77
78 TEST(bytesToHex, Empty) {
79     EXPECT_EQ(bytesToHex({}), "");
80 }
81
82 TEST(bytesToHex, SingleByte) {
83     EXPECT_EQ(bytesToHex({0x0A}), "0A ");
84     EXPECT_EQ(bytesToHex({0xFF}), "FF ");
85 }
86
87 TEST(bytesToHex, MultipleBytes) {
88     EXPECT_EQ(bytesToHex({0x0A, 0x12, 0x80, 0xAF}), "0A 12 80 AF ");
89 }
90
91 TEST(bytesToHex, ZeroPadding) {
92     EXPECT_EQ(bytesToHex({0x00, 0x01, 0x0F}), "00 01 0F ");
93 }
94
95 TEST(bytesToHex, UppercaseHex) {
96     EXPECT_EQ(bytesToHex({0xAB, 0xCD, 0xEF}), "AB CD EF ");
97 }
98
99 TEST(Roundtrip, BytesToHexToBytes) {
100     Bytes original{0x1B, 0x01, 0x00, 0x00, 0x00, 0x42};
101     EXPECT_EQ(hexToBytes(bytesToHex(original)), original);
102 }
103
104 TEST(Roundtrip, HexToBytesToHex) {
105     std::string normalized = "1B 01 00 00 00 42 ";
106     EXPECT_EQ(bytesToHex(hexToBytes(normalized)), normalized);
107 }

```

```
1 #pragma once
2
3 #include <InstructionPrototype.hpp>
4 #include <stdint>
5 #include <optional>
6 #include <vector>
7
8 class Instruction {
9 public:
10     const InstructionPrototype &prototype;
11     std::vector<std::uint8_t> bytes;
12
13     Instruction(
14         const InstructionPrototype &prototype, std::vector<std::uint8_t> bytes
15     );
16
17     std::string toAsm() const;
18     std::vector<std::uint8_t> getOperandBytes(int operandIndex) const;
19 };
```

```
1 #include "Instruction.hpp"
2
3 Instruction::Instruction(
4     const InstructionPrototype &prototype, std::vector<std::uint8_t> bytes
5 ) :
6     prototype(prototype), bytes(bytes) {}
7
8 std::string Instruction::toAsm() const {
9     std::string result = prototype.format;
10    std::size_t cumulativeBitOffset = prototype.opcodeBitWidth;
11
12    for (std::size_t i = 0; i < prototype.operands.size(); i++) {
13        std::string operandWildcard = '$' + std::to_string(i + 1);
14        std::size_t position = result.find(operandWildcard);
15        bool isRegisterInLowerNibble = cumulativeBitOffset % 8 != 0;
16
17        if (position == std::string::npos) {
18            break;
19        }
20
21        result.replace(
22            position,
23            operandWildcard.length(),
24            formatOperandBytes(
25                getOperandBytes(i),
26                prototype.operands[i],
27                isRegisterInLowerNibble
28            )
29        );
30
31        cumulativeBitOffset += getOperandBitWidth(prototype.operands[i]);
32    }
33
34    return result;
35 }
36
37 std::vector<std::uint8_t> Instruction::getOperandBytes(int operandIndex) const {
38     std::size_t offsetBits = prototype.opcodeBitWidth;
39
40     for (int i = 0; i < operandIndex; i++) {
41         offsetBits += getOperandBitWidth(prototype.operands[i]);
42     }
43
44     std::size_t operandBits =
45         getOperandBitWidth(prototype.operands[operandIndex]);
46     std::size_t operandBytes = (operandBits + 7) / 8;
47     std::vector<std::uint8_t> result(operandBytes);
48
49     for (std::size_t i = 0; i < operandBytes; i++) {
50         std::size_t bitIndex = offsetBits + i * 8;
51         std::size_t byteIndex = bitIndex / 8;
52
53         result[i] = bytes[byteIndex];
54     }
55 }
```

```
56     return result;
57 }
```

```
1 #include "Instruction.hpp"
2
3 #include <gtest/gtest.h>
4
5 using Bytes = std::vector<std::uint8_t>;
6
7 TEST/Instruction, ConstructorStoresPrototypeAndBytes) {
8     InstructionPrototype proto(
9         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
10    );
11    Instruction instr(proto, {0x0A, 0x12});
12
13    EXPECT_EQ(&instr.prototype, &proto);
14    EXPECT_EQ(instr.bytes, (Bytes{0x0A, 0x12}));
15 }
16
17 TEST/Instruction, ToAsmTwoRegsEightBitOpcode) {
18     InstructionPrototype proto(
19         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
20    );
21    Instruction instr(proto, {0x0A, 0x12});
22
23    EXPECT_EQ(instr.toAsm(), "SUB R1, R2");
24 }
25
26 TEST/Instruction, ToAsmHighAndLowNibbleRegs) {
27     InstructionPrototype proto(
28         "CMP $1, $2", 0x80, 8, {OperandType::Register, OperandType::Register}
29    );
30    Instruction instr(proto, {0x80, 0xAF});
31
32    EXPECT_EQ(instr.toAsm(), "CMP R10, R15");
33 }
34
35 TEST/Instruction, ToAsmRegAddrTwelveBitOpcode) {
36     InstructionPrototype proto(
37         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
38    );
39    Instruction instr(proto, {0x1B, 0x01, 0x00, 0x00, 0x00, 0x42});
40
41    EXPECT_EQ(instr.toAsm(), "MOV R1, [0x00000042]");
42 }
43
44 TEST/Instruction, ToAsmSwappedOperandOrderInFormat) {
45     InstructionPrototype proto(
46         "MOV $2, $1", 0x1B1, 12, {OperandType::Register, OperandType::Address}
47    );
48    Instruction instr(proto, {0x1B, 0x12, 0x00, 0x00, 0x00, 0x42});
49
50    EXPECT_EQ(instr.toAsm(), "MOV [0x00000042], R2");
51 }
52
53 TEST/Instruction, ToAsmThreeRegs) {
54     InstructionPrototype proto(
55         "ADD $1, $2, $3",
56         0x030,
```

```

57         12,
58         {OperandType::Register, OperandType::Register, OperandType::Register}
59     );
60     Instruction instr(proto, {0x03, 0x01, 0x23});
61
62     EXPECT_EQ(instr.toAsm(), "ADD R1, R2, R3");
63 }
64
65 TEST(Instruction, ToAsmLiteral16Positive) {
66     InstructionPrototype proto(
67         "MOV $1, $2", 0x1C1, 12, {OperandType::Register, OperandType::Literal16}
68     );
69     Instruction instr(proto, {0x1C, 0x11, 0x33, 0x44});
70
71     EXPECT_EQ(instr.toAsm(), "MOV R1, 13124");
72 }
73
74 TEST(Instruction, ToAsmLiteral16Negative) {
75     InstructionPrototype proto(
76         "MOV $1, $2", 0x1C1, 12, {OperandType::Register, OperandType::Literal16}
77     );
78     Instruction instr(proto, {0x1C, 0x1F, 0xFF, 0xFF});
79
80     EXPECT_EQ(instr.toAsm(), "MOV R15, -1");
81 }
82
83 TEST(Instruction, ToAsmLiteral32Positive) {
84     InstructionPrototype proto(
85         "MOV $1, $2", 0x1C2, 12, {OperandType::Register, OperandType::Literal32}
86     );
87     Instruction instr(proto, {0x1C, 0x21, 0x33, 0x44, 0x55, 0x66});
88
89     EXPECT_EQ(instr.toAsm(), "MOV R1, 860116326");
90 }
91
92 TEST(Instruction, ToAsmLiteral32Negative) {
93     InstructionPrototype proto(
94         "MOV $1, $2", 0x1C2, 12, {OperandType::Register, OperandType::Literal32}
95     );
96     Instruction instr(proto, {0x1C, 0x2F, 0xFF, 0xFF, 0xFF, 0xFF});
97
98     EXPECT_EQ(instr.toAsm(), "MOV R15, -1");
99 }
100
101 TEST(Instruction, GetOperandBytesRegsShareSameByte) {
102     // 8-bit opcode + two 4-bit regs: both operands live in byte 1
103     InstructionPrototype proto(
104         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
105     );
106     Instruction instr(proto, {0x0A, 0x12});
107
108     EXPECT_EQ(instr.getOperandBytes(0), Bytes{0x12});
109     EXPECT_EQ(instr.getOperandBytes(1), Bytes{0x12});
110 }
111
112 TEST(Instruction, GetOperandBytesRegThenAddress) {
113     InstructionPrototype proto(
114         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
115     );
116     Instruction instr(proto, {0x1B, 0x01, 0x00, 0x00, 0x00, 0x42});

```

```
117
118     EXPECT_EQ(instr.getOperandBytes(0), Bytes{0x01});
119     EXPECT_EQ(instr.getOperandBytes(1), (Bytes{0x00, 0x00, 0x00, 0x42}));
120 }
```

```
1 #pragma once
2
3 #include <Operand.hpp>
4 #include <cstdint>
5 #include <string>
6 #include <vector>
7
8 class InstructionPrototype {
9 public:
10     const std::string format;
11     const std::uint16_t opcode;
12     const std::uint8_t opcodeBitWidth;
13     const std::vector<OperandType> operands;
14
15     std::size_t size() const;
16
17     InstructionPrototype(
18         std::string format,
19         std::uint16_t opcode,
20         std::uint8_t opcodeBitWidth,
21         std::vector<OperandType> operands = {}
22     );
23
24     bool doesMatch(const std::vector<std::uint8_t> &bytes) const;
25 };
26
27 const std::vector<InstructionPrototype> INSTRUCTION_PROTOTYPES = {
28     InstructionPrototype(
29         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
30     ),
31     InstructionPrototype(
32         "MOV $2, $1", 0x1B1, 12, {OperandType::Register, OperandType::Address}
33     ),
34     InstructionPrototype(
35         "ADD $1, $2, $3",
36         0x030,
37         12,
38         {OperandType::Register, OperandType::Register, OperandType::Register}
39     ),
40     InstructionPrototype(
41         "ADD $1, $2, $3",
42         0x04,
43         8,
44         {OperandType::Register, OperandType::Register, OperandType::Address}
45     ),
46     InstructionPrototype(
47         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
48     ),
49     InstructionPrototype(
50         "SUB $1, $2", 0x0B0, 12, {OperandType::Register, OperandType::Address}
51     ),
52     InstructionPrototype(
53         "CMP $1, $2", 0x80, 8, {OperandType::Register, OperandType::Register}
54     ),
55     InstructionPrototype(
56         "MOV $1, $2", 0x1C1, 12, {OperandType::Register, OperandType::Literal16}
```

```
57     ),
58     InstructionPrototype(
59         "MOV $1, $2", 0x1C2, 12, {OperandType::Register, OperandType::Literal32}
60     ),
61 };
```

```
1 #include "InstructionPrototype.hpp"
2
3 std::size_t InstructionPrototype::size() const {
4     std::size_t sizeBits = opcodeBitWidth;
5
6     for (const auto &operand : operands) {
7         sizeBits += getOperandBitWidth(operand);
8     }
9
10    return (sizeBits + 7) / 8;
11 }
12
13 InstructionPrototype::InstructionPrototype(
14     std::string format,
15     std::uint16_t opcode,
16     std::uint8_t opcodeSize,
17     std::vector<OperandType> operands
18 ) :
19     format(format),
20     opcode(opcode),
21     opcodeBitWidth(opcodeSize),
22     operands(operands) {}
23
24 bool InstructionPrototype::doesMatch(const std::vector<std::uint8_t> &bytes
25 ) const {
26     if (bytes.size() != size()) {
27         return false;
28     }
29
30     std::size_t opcodeBytes = (opcodeBitWidth + 7) / 8;
31     const std::size_t backshift = opcodeBytes * 8 - opcodeBitWidth;
32     std::uint16_t extractedOpcode = 0;
33
34     for (std::size_t i = 0; i < opcodeBytes; i++) {
35         extractedOpcode <<= 8;
36         extractedOpcode |= bytes[i];
37     }
38
39     extractedOpcode >>= backshift;
40
41     if (extractedOpcode != opcode) {
42         return false;
43     }
44
45     return true;
46 }
```

```
1 #include "InstructionPrototype.hpp"
2
3 #include <gtest/gtest.h>
4
5 TEST(InstructionPrototype, ConstructorStoresFields) {
6     InstructionPrototype proto(
7         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
8     );
9
10    EXPECT_EQ(proto.format, "MOV $1, $2");
11    EXPECT_EQ(proto.opcode, 0x1B0);
12    EXPECT_EQ(proto.opcodeBitWidth, 12);
13    EXPECT_EQ(proto.operands.size(), 2);
14 }
15
16 TEST(InstructionPrototype, SizeEightBitOpcodeTwoRegs) {
17     // 8 + 4 + 4 = 16 bits = 2 bytes
18     InstructionPrototype proto(
19         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
20     );
21
22     EXPECT_EQ(proto.size(), 2);
23 }
24
25 TEST(InstructionPrototype, SizeTwelveBitOpcodeWithAddress) {
26     // 12 + 4 + 32 = 48 bits = 6 bytes
27     InstructionPrototype proto(
28         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
29     );
30
31     EXPECT_EQ(proto.size(), 6);
32 }
33
34 TEST(InstructionPrototype, SizeTwelveBitOpcodeThreeRegs) {
35     // 12 + 4 + 4 + 4 = 24 bits = 3 bytes
36     InstructionPrototype proto(
37         "ADD $1, $2, $3",
38         0x030,
39         12,
40         {OperandType::Register, OperandType::Register, OperandType::Register}
41     );
42
43     EXPECT_EQ(proto.size(), 3);
44 }
45
46 TEST(InstructionPrototype, SizeWithLit16) {
47     // 12 + 4 + 16 = 32 bits = 4 bytes
48     InstructionPrototype proto(
49         "MOV $1, $2", 0x1C1, 12, {OperandType::Register, OperandType::Literal16}
50     );
51
52     EXPECT_EQ(proto.size(), 4);
53 }
54
55 TEST(InstructionPrototype, SizeWithLit32) {
56     // 12 + 4 + 32 = 48 bits = 6 bytes
```

```

57     InstructionPrototype proto(
58         "MOV $1, $2", 0x1C2, 12, {OperandType::Register, OperandType::Literal32}
59     );
60
61     EXPECT_EQ(proto.size(), 6);
62 }
63
64 TEST(InstructionPrototype, DoesMatchEightBitOpcode) {
65     InstructionPrototype proto(
66         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
67     );
68
69     EXPECT_TRUE(proto.matches({0x0A, 0x12}));
70 }
71
72 TEST(InstructionPrototype, DoesMatchTwelveBitOpcode) {
73     InstructionPrototype proto(
74         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
75     );
76
77     EXPECT_TRUE(proto.matches({0x1B, 0x01, 0x00, 0x00, 0x00, 0x42}));
78 }
79
80 TEST(InstructionPrototype, DoesNotMatchWrongOpcode) {
81     InstructionPrototype proto(
82         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
83     );
84
85     EXPECT_FALSE(proto.matches({0x0B, 0x12}));
86 }
87
88 TEST(InstructionPrototype, DoesNotMatchWrongLength) {
89     InstructionPrototype proto(
90         "SUB $1, $2", 0x0A, 8, {OperandType::Register, OperandType::Register}
91     );
92
93     EXPECT_FALSE(proto.matches({0x0A, 0x12, 0x00}));
94     EXPECT_FALSE(proto.matches({0x0A}));
95 }
96
97 TEST(InstructionPrototype, DistinguishesTwelveBitNibbles) {
98     InstructionPrototype proto1B0(
99         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
100     );
101     InstructionPrototype proto1B1(
102         "MOV $2, $1", 0x1B1, 12, {OperandType::Register, OperandType::Address}
103     );
104
105     std::vector<std::uint8_t> bytes{0x1B, 0x01, 0x00, 0x00, 0x00, 0x42};
106
107     EXPECT_TRUE(proto1B0.matches(bytes));
108     EXPECT_FALSE(proto1B1.matches(bytes));
109 }
110
111 TEST(InstructionPrototype, MatchIgnoresOperandNibble) {
112     // The low nibble of byte 1 (0xF here) is the /reg operand, not part of
113     // opcode
114     InstructionPrototype proto(
115         "MOV $1, $2", 0x1B0, 12, {OperandType::Register, OperandType::Address}
116     );

```

```
117
118     EXPECT_TRUE(proto.matches({0x1B, 0x0F, 0x00, 0x00, 0x00, 0x42}));
119 }
120
121 TEST(InstructionPrototype, RegistryContainsNineVariantSixTemplates) {
122     EXPECT_EQ(INSTRUCTION_PROTOTYPES.size(), 9);
123 }
```

```
1  #pragma once
2
3  #include <stdint>
4  #include <stdexcept>
5  #include <string>
6  #include <vector>
7
8  enum class OperandType : std::uint8_t {
9      Register,
10     Address,
11     Literal16,
12     Literal32,
13 };
14
15 std::string formatOperandBytes(
16     const std::vector<std::uint8_t> &bytes,
17     OperandType type,
18     bool isRegisterInLowerNibble = true // I hate this
19 );
20
21 std::string formatRegister(
22     std::uint8_t byte, bool isRegisterInLowerNibble = true
23 );
24
25 std::string formatAddress(const std::vector<std::uint8_t> &bytes);
26
27 std::string formatLiteral16(const std::vector<std::uint8_t> &bytes);
28
29 std::string formatLiteral32(const std::vector<std::uint8_t> &bytes);
30
31 std::uint8_t getOperandBitWidth(OperandType type);
```

```
1  #include "Operand.hpp"
2
3  #include <bit>
4  #include <format>
5
6  std::string formatOperandBytes(
7      const std::vector<std::uint8_t> &bytes,
8      OperandType type,
9      bool isRegisterInLowerNibble
10 ) {
11     switch (type) {
12         case OperandType::Register:
13             return formatRegister(bytes[0], isRegisterInLowerNibble);
14         case OperandType::Address:
15             return formatAddress(bytes);
16         case OperandType::Literal16:
17             return formatLiteral16(bytes);
18         case OperandType::Literal32:
19             return formatLiteral32(bytes);
20         default:
21             throw std::invalid_argument("Unknown operand type");
22     }
23 }
24
25 std::string formatRegister(std::uint8_t byte, bool isRegisterInLowerNibble) {
26     if (!isRegisterInLowerNibble) {
27         byte >>= 4;
28     }
29
30     return std::format("R{}", byte & 0x0F);
31 }
32
33 std::string formatAddress(const std::vector<std::uint8_t> &bytes) {
34     std::uint32_t address =
35         (std::uint32_t(bytes[0]) << 24) | (std::uint32_t(bytes[1]) << 16) |
36         (std::uint32_t(bytes[2]) << 8) | std::uint32_t(bytes[3]);
37
38     return std::format("[0x{:08X}]", address);
39 }
40
41 std::string formatLiteral16(const std::vector<std::uint8_t> &bytes) {
42     std::uint16_t raw =
43         (std::uint16_t(bytes[0]) << 8) | std::uint16_t(bytes[1]);
44
45     return std::format("{} ", std::bit_cast<std::int16_t>(raw));
46 }
47
48 std::string formatLiteral32(const std::vector<std::uint8_t> &bytes) {
49     std::uint32_t raw =
50         (std::uint32_t(bytes[0]) << 24) | (std::uint32_t(bytes[1]) << 16) |
51         (std::uint32_t(bytes[2]) << 8) | std::uint32_t(bytes[3]);
52
53     return std::format("{} ", std::bit_cast<std::int32_t>(raw));
54 }
55
56 std::uint8_t getOperandBitWidth(OperandType type) {
```

```
57     switch (type) {
58         case OperandType::Register:
59             return 4;
60         case OperandType::Address:
61             return 32;
62         case OperandType::Literal16:
63             return 16;
64         case OperandType::Literal32:
65             return 32;
66         default:
67             throw std::invalid_argument("Unknown operand type");
68     }
69 }
```

```
1 #include "Operand.hpp"
2
3 #include <gtest/gtest.h>
4
5 TEST(getOperandBitWidth, AllVariants) {
6     EXPECT_EQ(getOperandBitWidth(OperandType::Register), 4);
7     EXPECT_EQ(getOperandBitWidth(OperandType::Address), 32);
8     EXPECT_EQ(getOperandBitWidth(OperandType::Literal16), 16);
9     EXPECT_EQ(getOperandBitWidth(OperandType::Literal32), 32);
10 }
11
12 TEST(formatRegister, LowerNibble) {
13     EXPECT_EQ(formatRegister(0x05, true), "R5");
14     EXPECT_EQ(formatRegister(0x0F, true), "R15");
15     EXPECT_EQ(formatRegister(0x00, true), "R0");
16 }
17
18 TEST(formatRegister, UpperNibble) {
19     EXPECT_EQ(formatRegister(0x50, false), "R5");
20     EXPECT_EQ(formatRegister(0xF0, false), "R15");
21     EXPECT_EQ(formatRegister(0x00, false), "R0");
22 }
23
24 TEST(formatRegister, IgnoresUnrelatedNibble) {
25     EXPECT_EQ(formatRegister(0xAB, true), "R11");
26     EXPECT_EQ(formatRegister(0xAB, false), "R10");
27 }
28
29 TEST(formatAddress, Zero) {
30     EXPECT_EQ(formatAddress({0x00, 0x00, 0x00, 0x00}), "[0x00000000]");
31 }
32
33 TEST(formatAddress, BigEndianAssembly) {
34     EXPECT_EQ(formatAddress({0x00, 0xC0, 0x00, 0xA1}), "[0x00C000A1]");
35     EXPECT_EQ(formatAddress({0xDE, 0xAD, 0xBE, 0xEF}), "[0xDEADBEEF]");
36 }
37
38 TEST(formatAddress, AllOnes) {
39     EXPECT_EQ(formatAddress({0xFF, 0xFF, 0xFF, 0xFF}), "[0xFFFFFFFF]");
40 }
41
42 TEST(formatLiteral16, Zero) {
43     EXPECT_EQ(formatLiteral16({0x00, 0x00}), "0");
44 }
45
46 TEST(formatLiteral16, PositiveValues) {
47     EXPECT_EQ(formatLiteral16({0x00, 0x01}), "1");
48     EXPECT_EQ(formatLiteral16({0x33, 0x44}), "13124");
49     EXPECT_EQ(formatLiteral16({0x7F, 0xFF}), "32767");
50 }
51
52 TEST(formatLiteral16, NegativeValues) {
53     EXPECT_EQ(formatLiteral16({0xFF, 0xFF}), "-1");
54     EXPECT_EQ(formatLiteral16({0x80, 0x00}), "-32768");
55     EXPECT_EQ(formatLiteral16({0xFF, 0x80}), "-128");
56 }
```

```

57
58 TEST(formatLiteral32, Zero) {
59     EXPECT_EQ(formatLiteral32({0x00, 0x00, 0x00, 0x00}), "0");
60 }
61
62 TEST(formatLiteral32, PositiveValues) {
63     EXPECT_EQ(formatLiteral32({0x00, 0x00, 0x00, 0x01}), "1");
64     EXPECT_EQ(formatLiteral32({0x33, 0x44, 0x55, 0x66}), "860116326");
65     EXPECT_EQ(formatLiteral32({0x7F, 0xFF, 0xFF, 0xFF}), "2147483647");
66 }
67
68 TEST(formatLiteral32, NegativeValues) {
69     EXPECT_EQ(formatLiteral32({0xFF, 0xFF, 0xFF, 0xFF}), "-1");
70     EXPECT_EQ(formatLiteral32({0x80, 0x00, 0x00, 0x00}), "-2147483648");
71     EXPECT_EQ(formatLiteral32({0xFF, 0xFF, 0xFF, 0x80}), "-128");
72 }
73
74 TEST(formatOperandBytes, DispatchesToRegister) {
75     EXPECT_EQ(formatOperandBytes({0x0A}, OperandType::Register, true), "R10");
76     EXPECT_EQ(formatOperandBytes({0xA0}, OperandType::Register, false), "R10");
77 }
78
79 TEST(formatOperandBytes, DispatchesToAddress) {
80     EXPECT_EQ(
81         formatOperandBytes({0x00, 0x00, 0x00, 0x42}, OperandType::Address),
82         "[0x00000042]"
83     );
84 }
85
86 TEST(formatOperandBytes, DispatchesToLiteral16) {
87     EXPECT_EQ(formatOperandBytes({0xFF, 0xFF}, OperandType::Literal16), "-1");
88 }
89
90 TEST(formatOperandBytes, DispatchesToLiteral32) {
91     EXPECT_EQ(
92         formatOperandBytes({0x00, 0x00, 0x00, 0x01}, OperandType::Literal32),
93         "1"
94     );
95 }
96
97 TEST(formatOperandBytes, ThrowsOnUnknownType) {
98     auto bogus = static_cast<OperandType>(0xFF);
99     EXPECT_THROW(formatOperandBytes({0x00}, bogus), std::invalid_argument);
100 }

```

```
1  #include <gtest/gtest.h>
2
3  #include <Decompiler.hpp>
4  #include <Hex.hpp>
5  #include <Instruction.hpp>
6  #include <string>
7  #include <vector>
8
9  namespace {
10
11  std::string decodeSingle(std::string_view hex) {
12      auto bytes = hexToBytes(hex);
13      auto instructions = decompile(bytes);
14      return instructions[0].toAsm();
15  }
16
17  std::vector<std::string> decodeAll(std::string_view hex) {
18      auto bytes = hexToBytes(hex);
19      auto instructions = decompile(bytes);
20
21      std::vector<std::string> result;
22      for (const auto &instr : instructions) {
23          result.push_back(instr.toAsm());
24      }
25
26      return result;
27  }
28
29 } // namespace
30
31 TEST(Variant6, Template2_MovRegFromAddr) {
32     EXPECT_EQ(decodeSingle("1B 01 00 00 00 42"), "MOV R1, [0x00000042]");
33 }
34
35 TEST(Variant6, Template3_MovAddrFromReg) {
36     EXPECT_EQ(decodeSingle("1B 12 00 00 00 42"), "MOV [0x00000042], R2");
37 }
38
39 TEST(Variant6, Template6_AddRegRegReg) {
40     EXPECT_EQ(decodeSingle("03 01 23"), "ADD R1, R2, R3");
41 }
42
43 TEST(Variant6, Template7_AddRegRegAddr) {
44     EXPECT_EQ(decodeSingle("04 12 00 00 00 42"), "ADD R1, R2, [0x00000042]");
45 }
46
47 TEST(Variant6, Template8_SubRegReg) {
48     EXPECT_EQ(decodeSingle("0A 12"), "SUB R1, R2");
49 }
50
51 TEST(Variant6, Template9_SubRegAddr) {
52     EXPECT_EQ(decodeSingle("0B 01 00 00 00 42"), "SUB R1, [0x00000042]");
53 }
54
55 TEST(Variant6, Template26_CmpRegReg) {
56     EXPECT_EQ(decodeSingle("80 AF"), "CMP R10, R15");
57 }
```

```

57 }
58
59 TEST(Variant6, Template28_MovRegLit16Positive) {
60     EXPECT_EQ(decodeSingle("1C 11 33 44"), "MOV R1, 13124");
61 }
62
63 TEST(Variant6, Template28_MovRegLit16Negative) {
64     EXPECT_EQ(decodeSingle("1C 1F FF FF"), "MOV R15, -1");
65 }
66
67 TEST(Variant6, Template29_MovRegLit32Positive) {
68     EXPECT_EQ(decodeSingle("1C 21 33 44 55 66"), "MOV R1, 860116326");
69 }
70
71 TEST(Variant6, Template29_MovRegLit32Negative) {
72     EXPECT_EQ(decodeSingle("1C 2F FF FF FF FF"), "MOV R15, -1");
73 }
74
75 TEST(Variant6, AllTemplatesInSingleStream) {
76     auto asm_lines = decodeAll(
77         "1B 01 00 00 00 42 " // template 2 : MOV R1, [0x00000042]
78         "1B 12 00 00 00 42 " // template 3 : MOV [0x00000042], R2
79         "03 01 23 " // template 6 : ADD R1, R2, R3
80         "04 12 00 00 00 42 " // template 7 : ADD R1, R2, [0x00000042]
81         "0A 12 " // template 8 : SUB R1, R2
82         "0B 01 00 00 00 42 " // template 9 : SUB R1, [0x00000042]
83         "80 AF " // template 26 : CMP R10, R15
84         "1C 11 33 44 " // template 28 : MOV R1, 13124
85         "1C 21 33 44 55 66" // template 29 : MOV R1, 860116326
86     );
87
88     EXPECT_EQ(
89         asm_lines,
90         (std::vector<std::string>{
91             "MOV R1, [0x00000042]",
92             "MOV [0x00000042], R2",
93             "ADD R1, R2, R3",
94             "ADD R1, R2, [0x00000042]",
95             "SUB R1, R2",
96             "SUB R1, [0x00000042]",
97             "CMP R10, R15",
98             "MOV R1, 13124",
99             "MOV R1, 860116326",
100         })
101     );
102 }

```